Chapter 1 Fundamental of Concepts in Graph Theory

1.1 Graphs

1.1.1

Definition 1.1.1.1

- (Directed Graph)
- (Undirected Grpah)

1.1.2 Vertexes

Definition 1.1.2.1 ...

1.1.3 Matrix Representation

1.2 Path, Circle and Connectedness

Definition 1.2.0.1

- A walk in a graph G is a sequence as following:

  ...

  where $e_i = (x_i, x_{i+1})$ or $e_i = \{x_i, x_{i+1}\}$
- A **path**
- A **cycle** in a graph G is a

1.2.1 Connectedness

Undirected Graph

◇ **Lemma 1.2.1.1** Suppose G

∘ **Proof** ... □

Definition 1.2.1.2 ...

♠ **Proposition 1.2.1.3** ...

∘ **Proof** Take the negation G is disconnected⇔ There exists some collection X such that □

Then, the component for graphs is well-defined

Definition 1.2.1.4 Let G be a undirected graph, then,

$$V(G) = \cup V(G_i)$$

Directed Graph

♠ **Proposition 1.2.1.5** ...

∘ **Proof** ... □

Definition 1.2.1.6 ...

Euler Tour

Definition 1.2.1.7

-
-

♠ **Proposition 1.2.1.8** A connected graph has one and only one Eulertour, when the graph is Euler.

∘ **Proof**

- ⇒
- ⇐

□

1.2.2 Leaf,Tree, Forest

Tree, Leaves

Definition 1.2.2.1 Suppose following graphs are undirected.

- (Tree) A
  - Connected
  - Undirected
  - with no cycle

  graph is called **tree**.
- The vertice of grad 1 is called **leaf**.

◇ **Lemma 1.2.2.2** The following proposition holds: For undirected graph G, we have

- Every tree with at least two vertices contains at least 2 leaves;
- Suppose G be a tree with
  - n Vertices
  - m edges

– p connected components.

Then we have

$$n = m + p$$

∘ **Proof**

-
-

□

♠ **Proposition 1.2.2.3** The followings are equivalent:

-
-
-
-
-
-
-

∘ **Proof** ... □

Arborenzence

Definition 1.2.2.4 ...

Minimal Spanned Tree

Definition 1.2.2.5 ...

1.2.3 Graph

Chapter 2 Optimal Tree and Path

2.1 Minimum Spanning Tree Problem

Kruskals Algorithms

Prim Algorithms

2.2 Graph Traversal

Definition 2.2.0.1

- A **connected and undirected graph**, which contains no circle, is called **tree**.
- A **undirected graph**, which contains no circle, is called **forest**;
- The vertex e, such that $|\delta(e)| = 1$, is called **leaf**,representing endpoints.

◇ **Lemma 2.2.0.2** The following propositions hold:

- Every tree with at least 2 vertexes contains 2 leaves;
- Suppose G be a forest with n vertexes, m edges and p components. Then:

$$n = m + p$$

∘ **Proof** ... □

♠ **Proposition 2.2.0.3** Suppose G be a undirected graph with n vertexes. The following are equivalent

- G is a tree;
- G has $n - 1$ vertexes and contains no circle;
- G has $n - 1$ vertexes and is connected;
- G is connected and by deleting a vertex, we get a disconnected graph;
- G is a graph with $\delta(X) \neq \varnothing$, for every $\varnothing \subseteq X \subseteq V(G)$, and the property is destroyed by deleting one vertex;
- G is a forest.
- Every two path in G is connected by a unique path;

∘ **Proof** ... □

Definition 2.2.0.4

- A directed graph G is called **arborescence**, if:
  - At every vertex $e \in E(G)$, the $\delta^+(e) \leq 1$;
  - The correspondent undirected graph is a tree.

- In an arborescence, there exists one and only one of vertex such that $\delta^-(r) = \varnothing$, which is called the **root**;
- As a dual of concept forest, such directed graph is called **Silvanescence**.

Graph Traversal Algorithm

♠ **Proposition 2.2.0.5** ...

∘ **Proof** ... □

DFS

BFS

Distance

Definition 2.2.0.6 (Unweighted) For two vertexes x, y in a graph G, the distance $dist(x, y)$ is the minimal number of $x - y$ path.
If there exists no $x - y$ path, then $dist(x, y) := \infty$.

BFS,DFS

2.3 Weighted Graph

Definition 2.3.0.1 A **weighted graph** is a graph G with a function

$$c : E(G) \to \mathbb{R}$$

which is called the **weight of edge**.

Let $H \subseteq G$ be a subgraph. Then,

$$c(E(H)) := \sum_{e \in E(H)} c(e)$$

is called the **weight** of subgraph H.

Distance

A main problem is: **The shortest path problem**.

Dijkstra's Algorithm

Definition 2.3.0.2

- Input: Weighted graph (G, c) with $s \in V(G)$.
- Output: Arboriculture $A = (R, T)$ in G, such that R contains all , and

$$disc_{(G,c)}(s, v) = disc_{(A,c)}(s, v); \forall v \in R$$

The Pseudocode is given here: 1

Algorithm 1 Dijkstra's Algorithm

Require: Weighted Graph (G, c), Knot $s \in V(G)$;
Ensure: Arborescences $A = (R, T)$ in G, which contains all , and satisfy $dist_{G,c}(s, v) = dist_{(A,c)}(s, v)$

```
R← 0;Q← {s} ;l(s)← 0        \\ R:The ensured vertex with shortest path; Q:
while Q≠ ∅: do
    Select v ∈ Q with l(v) smallest;
    Q← Q − {v};R↔ R − {v};
    \\ To find out the neighbors of v;
    for e = (v, w) ∈ δ⁺_G(v) with w ∉ R do
        if w ∉ Q or l(v) + c(e) < l(w) then
            l(w) ← l(v) + c(e); p(w) ← e; Q ← Q ∪ {w};        \\ If we find a shorter path from s to w via v,
            \\ then:l(w) := l(v) + c(e); p(w) = e, the previous edge; Q := Q ∪ {w}, extending the
        end if
    end for
end while
T ← {p(v)|v ∈ R − {s}};
```

♠ **Proposition 2.3.0.3** Dijkstras Algorithmus is correct. And has a

$$\mathcal{O}(n^2 + m)$$

where $n = |V(G)|, m = |E(G)|$.

∘ **Proof** ... □

Remark
Remark

Definition 2.3.0.4 ...

Chapter 3 Flow in Network

3.1 Network

Definition 3.1.0.1 ...